

On Prem to Cloud DB MIGRATION SERIES

TECHNICAL STANDARD OPERATING PROCEDURE

Strictly use this as a TEMPLATE Only and do through Testing on Non-Prod Environment.

Zero-Downtime* Migration of a 4 TB SQL Server Database to AWS

Replicated Cutover Using Native Backup Seeding, AWS DMS CDC / Log Shipping, and a Rehearsed <15-Minute Cutover Window

SOP ID	DBA-SOP-MIG-AWS-4TB-001
Version	1.0
Classification	Cloud DB Admin Learning Series — Reference / Runbook
Source Platform	SQL Server 2019/2022 — On-Premises Always On AG
Target Platform	Amazon RDS for SQL Server (Multi-AZ) or EC2 Self-Managed AG
Downtime Budget	< 15 minutes (application write-outage window)
Author	Cloud DB Admin
Date	06 th August 2026 - Thursday

* "Zero-downtime" refers to elimination of extended read/write outage; a short, budgeted write-freeze is still required for transactional consistency.

Table of Contents

Table of Contents	2
Document Control.....	4
Scope of Applicability.....	4
Out of Scope	4
1. Real-Time Scenario.....	5
1.1 Environment.....	5
1.2 Why Backup-Restore-Over-the-Wire Is Ruled Out.....	5
2. Architecture Decision: RDS vs. EC2 Self-Managed	6
3. Prerequisites & Pre-Migration Checklist	7
3.1 AWS-Side Setup.....	7
3.2 Rehearsal Environment.....	7
3.3 Governance	7
4. Phase 1 — Seed the Target	8
4.1 Option B (Reference Path) — Log Shipping to EC2 Self-Managed AG	8
4.2 Option A — RDS via S3 Backup + DMS CDC.....	8
5. Phase 2 — Continuous Sync & Multi-Day Validation	10
6. Phase 3 — T-Minus 24 Hours	11
7. Phase 4 — Cutover Runbook (Minute-by-Minute)	12
8. Rollback Plan	13
8.1 Rollback Trigger Criteria.....	13
8.2 Rollback Procedure	13
9. Reconciliation Scripts.....	14
9.1 Row Count Comparison	14
9.2 Checksum Comparison (Per Table)	14
9.3 Log-Shipping / CDC Lag Snapshot	14
9.4 Automated Drift Alert (Daily During Phase 2)	14
10. Timing Budget Reference.....	15
11. Troubleshooting & Common Pitfalls	16
Log-shipping restore falls behind and never catches up.....	16
DMS CDC task shows high CDCLatencyTarget but low CDCLatencySource	16
Reconciliation checksum mismatch on a specific table only	16
Application still connects to old IP after DNS repoint.....	16
SQL Agent jobs on target fail immediately after cutover	16
12. Appendix — Supporting Scripts	17
12.1 Pre-Flight Connectivity Check (PowerShell)	17
12.2 Active Write-Transaction Check Before Freeze.....	17

12.3 Deny/Allow Connect Toggle (Freeze / Unfreeze).....	17
12.4 DMS CLI Snippets	17

Document Control

Version	Date	Author	Change Summary
1.0	06 th Aug 2026	Cloud DB Admin	Initial release — covers RDS (backup/restore + DMS CDC) and EC2 self-managed (log shipping) migration paths.

Scope of Applicability

This SOP applies to migrations of large (≥ 1 TB) OLTP SQL Server databases from on-premises or co-located infrastructure into AWS, where the business cannot tolerate an outage long enough for a conventional backup-and-restore-over-the-wire cutover. It assumes SQL Server 2019 or later on the source and either Amazon RDS for SQL Server or a self-managed EC2 SQL Server instance as the target.

Out of Scope

- Heterogeneous migrations to Aurora PostgreSQL / MySQL (schema and data-type conversion via SCT is a separate SOP).
- Databases under 200 GB, where a direct backup/restore over a Direct Connect or Snowball path may fit the downtime budget without CDC.
- Migrations requiring near-zero (sub-second) RPO with synchronous commit across regions — that requires a distributed Availability Group design, referenced separately.

1. Real-Time Scenario

The steps in this SOP are written against a concrete, representative scenario so that every command and threshold below is something you can rehearse literally, not just conceptually.

1.1 Environment

Item	Detail
Workload	OLTP order-management database, “ContosoOrders”, 4.1 TB data + 380 GB log churn/day
Source topology	On-prem Always On AG: VMSQL01 (Primary), VMSQL02 (Sync Secondary), VMSQL03 (Async DR)
Source version	SQL Server 2019 Enterprise, Windows Server 2019
Peak write rate	~2,800 batch requests/sec at business-hours peak; ~150/sec off-peak
Network	AWS Direct Connect, 1 Gbps dedicated, ~6 ms RTT to us-east-1
Target — Option A	Amazon RDS for SQL Server 2019/2022 Enterprise, db.r6i.4xlarge, Multi-AZ, io2 storage
Target — Option B	Self-managed EC2 Always On AG: EC2SQL01 / EC2SQL02 across two AZs, r6i.4xlarge, gp3/io2
Downtime budget	15 minutes maximum application write-outage, agreed with the business in the change record
Rollback SLA	Must be able to revert to the on-prem primary within the same 15-minute window if smoke tests fail

1.2 Why Backup-Restore-Over-the-Wire Is Ruled Out

A native full backup of a 4 TB database, even compressed at a conservative 2.5:1 ratio, is roughly 1.6 TB to move. Over a 1 Gbps Direct Connect link at realistic sustained throughput (~70–90 MB/s after protocol and compression overhead), that is 5–6+ hours just for the copy, before restore time, before catching up on log activity generated during the copy. That single fact drives the entire design: the target has to be seeded and kept current while the source stays live, so that cutover is a switch-of-pointer operation, not a copy operation.

DESIGN PRINCIPLE

Every minute of heavy lifting — backup, transfer, restore, initial sync — happens BEFORE the outage window opens. The outage window only ever contains: stop writes → drain → verify zero lag → reconcile → repoint → start → smoke test.

2. Architecture Decision: RDS vs. EC2 Self-Managed

Choose the path based on feature dependency, not just cost. Decide this before Phase 1 — the seeding mechanism differs.

Criterion	Option A — RDS for SQL Server	Option B — EC2 Self-Managed AG
Seeding method	Native backup to S3 → RDS restore via option group	Log shipping or Distributed AG directly to EC2
CDC catch-up tool	AWS DMS, CDC-only task against source AG listener	Native transaction log shipping (restore WITH NORECOVERY)
Max downtime achievable	2–5 minutes typical	1–3 minutes typical (native technology, less handoff)
SQL Agent jobs / linked servers / CLR	Restricted — RDS does not give OS/filesystem access	Full support — unrestricted sysadmin access
Patching / OS ownership	AWS-managed	Customer-managed
Cross-DB transactions, DBCC, trace flags	Limited set exposed via rdsadmin	Full native support
Best fit	Standardized workloads, lower ops overhead	Workloads dependent on Agent jobs, AG features, CLR, DAG

THIS SOP'S REFERENCE PATH

ContosoOrders depends on 14 SQL Agent jobs and 2 CLR assemblies — Option B (EC2 self-managed AG) is the path taken below. Option A steps are included at each phase as an alternate track where the mechanism differs materially (S3-backup seeding and DMS CDC).

3. Prerequisites & Pre-Migration Checklist

3.1 AWS-Side Setup

1. Provision target: EC2SQL01 and EC2SQL02 in separate AZs (or the RDS Multi-AZ instance), sized to match or exceed source vCPU/memory (r6i.4xlarge → 16 vCPU / 128 GB matched).
2. Provision storage: io2 Block Express volumes sized for 4 TB data + 30% growth headroom + separate log volume sustaining source's peak log write rate (measure via Perfmon 'Disk Bytes/sec' on VMSQL01 for 7 days).
3. Create an S3 bucket (contosoorders-migration-stage) with versioning enabled, SSE-KMS encryption, and a lifecycle rule to expire objects after 30 days.
4. Create an IAM role for SQL Server backup/restore to S3 (source EC2/on-prem instance profile or credential for BACKUP DATABASE ... TO URL).
5. Open Direct Connect / VPN routes: SQL port 1433 between VMSQL01/02/03 and EC2SQL01/02; confirm with a manual telnet/Test-NetConnection before Day 1.
6. Install and configure SQL Server (same version/edition/patch level as source, minimum) on EC2SQL01/EC2SQL02; align collation, max server memory, tempdb file count, and trace flags to source.
7. Pre-stage logins, SQL Agent jobs, linked servers, and CLR assemblies on the target via scripted DDL (do not rely on restore alone to carry server-level objects).

3.2 Rehearsal Environment

1. Stand up a clone: restore a recent full backup of ContosoOrders into an isolated VPC/subnet mirroring the production target sizing.
2. Replay production log traffic (or run a representative synthetic OLTP load, e.g. HammerDB) against the clone's source to simulate write volume during rehearsal.
3. Execute this entire SOP end-to-end against the clone, including the rollback procedure in Section 8, and record actual timings for each phase.
4. Do not proceed to a production change window until at least one full rehearsal has completed with cutover inside the 15-minute budget and rollback verified to work.

3.3 Governance

1. Raise the Change Record with the business-approved cutover window, downtime budget, and rollback trigger criteria (Section 8.1) attached.
2. Confirm application connection strings route through a DNS CNAME or a connection-string-config flag — never a hardcoded IP — so cutover is a repaint, not a redeploy.
3. Confirm freeze on schema changes (DDL) against ContosoOrders from T-minus 24 hours through cutover completion.

4. Phase 1 — Seed the Target

Goal: get a transactionally consistent full copy of ContosoOrders onto the target while the source stays fully online. This runs days ahead of cutover.

4.1 Option B (Reference Path) — Log Shipping to EC2 Self-Managed AG

Step 1. On VMSQL01, take a full native backup with checksum and compression, striped across multiple files to parallelize I/O:

```
BACKUP DATABASE [ContosoOrders]
TO DISK = N'\\BackupShare\ContosoOrders_Full_1.bak',
    DISK = N'\\BackupShare\ContosoOrders_Full_2.bak',
    DISK = N'\\BackupShare\ContosoOrders_Full_3.bak',
    DISK = N'\\BackupShare\ContosoOrders_Full_4.bak'
WITH COMPRESSION, CHECKSUM, STATS = 5;
```

Step 2. Copy the backup set to EC2SQL01 over Direct Connect (robocopy with /MT multithreading, or aws s3 sync if staging through S3).

```
robocopy \\BackupShare \\EC2SQL01\BackupStage ContosoOrders_Full_*.bak /MT:8 /Z /R:3
```

Step 3. Restore WITH NORECOVERY on EC2SQL01 to leave the database in a loading state ready for log chains:

```
RESTORE DATABASE [ContosoOrders]
FROM DISK = N'\\BackupStage\ContosoOrders_Full_1.bak',
    DISK = N'\\BackupStage\ContosoOrders_Full_2.bak',
    DISK = N'\\BackupStage\ContosoOrders_Full_3.bak',
    DISK = N'\\BackupStage\ContosoOrders_Full_4.bak'
WITH NORECOVERY, REPLACE, STATS = 5,
MOVE N'ContosoOrders_Data' TO N'D:\SQLData\ContosoOrders.mdf',
MOVE N'ContosoOrders_Log' TO N'L:\SQLLog\ContosoOrders.ldf';
```

Step 4. Configure a native log-shipping job (or a scripted loop) that ships and restores transaction-log backups from VMSQL01 to EC2SQL01 on a tight interval — 5 minutes for the first 48 hours, tightening to 1 minute as cutover approaches:

```
-- Source: scheduled Agent job every 1 min near cutover
BACKUP LOG [ContosoOrders]
TO DISK = N'\\BackupShare\LogShip\ContosoOrders_Log.trn'
WITH COMPRESSION, CHECKSUM, STATS = 25;

-- Target: restore loop, WITH NORECOVERY keeps DB ready for next log
RESTORE LOG [ContosoOrders]
FROM DISK = N'\\BackupStage\LogShip\ContosoOrders_Log.trn'
WITH NORECOVERY, STATS = 25;
```

Step 5. Repeat Steps 3–4 to seed EC2SQL02 as the AG secondary, then build the Always On AG between EC2SQL01/EC2SQL02 once both are in a restoring state and caught up.

4.2 Option A — RDS via S3 Backup + DMS CDC

1. BACKUP DATABASE ... TO URL pointing at the S3 staging bucket, using the SQL Server native backup-to-S3 capability (or the RDS 'Native Backup and Restore' option group feature).

2. On the RDS target, run `msdb.dbo.rds_restore_database` pointing at the S3 object; monitor with `rds_task_status` until COMPLETE.
3. Create an AWS DMS replication instance sized to at least match source IOPS headroom (`dms.r5.2xlarge` for this data volume).
4. Create source (SQL Server, CDC-enabled) and target (RDS) endpoints; enable CDC on ContosoOrders via `sys.sp_cdc_enable_db` and `sp_cdc_enable_table` on all in-scope tables.
5. Create a DMS task in 'Full load + CDC' initially against a small validation subset, then run a dedicated 'CDC only' task once the RDS target has been seeded from the S3 restore, using the backup's LSN as the CDC start point (`cdc_start_position`).

COMMON PITFALL

If CDC is enabled on the source AFTER the backup used to seed the target was taken, the LSN watermark won't align and DMS will either miss changes or replay changes already present. Always enable CDC first, capture the LSN, then take the seeding backup.

5. Phase 2 — Continuous Sync & Multi-Day Validation

Goal: let the target track the source for several days under real production load, proving stability before anyone talks about a cutover date.

1. Tighten the log-shipping restore interval to 1 minute (Option B) or confirm DMS CDC latency metric `CDCLatencySource / CDCLatencyTarget` stays under 60 seconds (Option A).
2. Build a lag-monitoring job that alerts if restore/CDC latency exceeds a threshold (2 minutes) for more than 5 consecutive minutes.

```
-- Log shipping lag check (run on monitoring server against msdb on EC2SQL01)
SELECT secondary_database AS database_name, last_copied_date AS backup_time,
last_restored_date AS restore_time, DATEDIFF(MINUTE, last_restored_date,
GETDATE()) AS restore_lag_minutes FROM msdb.dbo.log_shipping_monitor_secondary
WHERE secondary_database = 'ContosoOrders';
```

3. Run row-count and checksum reconciliation daily between source and target on the largest 10 tables (script in Section 9) and log the drift trend — it should trend to zero, not stay flat.
4. Validate that server-level objects on the target are current: logins, SQL Agent jobs, linked servers, CLR assemblies, certificates used for TDE/Always Encrypted, and SSIS/SSRS dependent objects if any.
5. Load-test the target independently (read-only queries against a snapshot) to confirm the sized instance meets production query-plan expectations — do this against a copy, never against the live sync target.
6. Run at least one full rehearsal of Phase 4 (cutover) and Section 8 (rollback) against the clone environment from Section 3.2, timing every step.

EXIT CRITERIA FOR PHASE 2

Minimum 5 consecutive days of sync with lag consistently under 2 minutes, zero unresolved reconciliation drift on validated tables, one successful timed rehearsal under the 15-minute budget, and rollback proven to work from the same rehearsal.

6. Phase 3 — T-Minus 24 Hours

1. Confirm Change Record approved, rollback window agreed, and all stakeholders (app owners, network, on-call DBA, business sign-off) on the bridge line for the cutover slot.
2. Freeze schema changes (DDL) against ContosoOrders on the source.
3. Re-run the full reconciliation script (Section 9) and confirm zero drift on all validated tables, not just the sample.
4. Confirm DNS TTL on the application connection CNAME is reduced to 30–60 seconds at least 24 hours in advance, so the repoint in Phase 4 propagates fast.
5. Pre-stage the cutover runbook checklist (Phase 4) with a dedicated executor and a timekeeper; dry-read it aloud on the bridge call the day before.
6. Confirm current lag is comfortably under 2 minutes and trending flat-to-zero, not climbing.
7. Snapshot/backup the source one final time immediately before the window opens, purely as a rollback artifact (not part of the sync chain).

7. Phase 4 — Cutover Runbook (Minute-by-Minute)

This is the only part of the process that happens inside the 15-minute downtime budget. Every action here has already been rehearsed in Section 3.2. Times are elapsed minutes:seconds from window start (T+00:00), sized to this scenario — pad or compress based on your rehearsal timings.

T+	Action	Owner	Exit Check
00:00	Announce start on bridge call. Put application into maintenance mode / stop write traffic at the app tier (feature flag or LB rule).	App Owner	Confirmed: 0 active write transactions via sp_whoisactive filter
00:30	On source, disable inbound app logins (DENY CONNECT or firewall rule) to prevent stray writes.	DBA	New connection attempts rejected
01:00	Force final log backup/shipment (Option B) or confirm DMS CDC has drained to source LSN (Option A).	DBA	Latest log restored on target / CDC latency = 0
02:30	Verify restore_lag_minutes = 0 and no pending log files in the ship queue.	DBA	Lag metric reads 0; queue empty
03:00	Run reconciliation script: row counts + checksums on all critical tables between source and target.	DBA	Zero drift on every validated table
05:30	Bring target database online: for Option B, RESTORE DATABASE WITH RECOVERY then join/promote AG; for Option A, confirm RDS DB is in available state.	DBA	Database reports ONLINE, read/write
06:30	Run post-recovery integrity check: DBCC CHECKDB WITH PHYSICAL_ONLY (fast check — full CHECKDB already run pre-window).	DBA	No corruption reported
08:00	Start dependent SQL Agent jobs on target; verify job history begins cleanly.	DBA	Jobs enabled, next run scheduled
08:30	Repoint application connection string / DNS CNAME to the target listener or RDS endpoint.	DBA / Network	nslookup confirms new IP; TTL respected
09:30	Run smoke test suite: login, order create, order read, payment stub, report query.	App Owner + DBA	All smoke tests pass
11:00	Re-enable write traffic at the app tier / exit maintenance mode.	App Owner	Live transactions flowing to target
12:00	Monitor active sessions, wait stats, and error log on target for 2–3 minutes of live traffic.	DBA	No blocking, no unexpected errors
14:30	Declare cutover complete on bridge call; keep source AG intact (do not tear down) for the agreed rollback watch period.	DBA Lead	Go/No-Go recorded; source retained

GO/NO-GO GATE AT T+03:00

If reconciliation shows any drift at T+03:00, STOP. Do not bring the target online. Extend the freeze, re-run the final log shipment, and re-check. Going over the 15-minute budget by a few minutes to protect data integrity is always the right call over cutting over with unverified data.

8. Rollback Plan

8.1 Rollback Trigger Criteria

- Reconciliation drift detected at the T+03:00 gate that cannot be resolved within 5 additional minutes.
- Smoke tests fail on 2 or more critical paths (login, order create, payment) at T+09:30.
- Unexpected blocking, corruption, or error-log severity ≥ 20 observed during the T+12:00 monitoring step.
- Any DBA or App Owner on the bridge call invokes rollback — no single-person veto required to trigger it; consensus is not required to protect data, only to declare success.

8.2 Rollback Procedure

1. Re-enable inbound connections on the source (reverse the DENY CONNECT from T+00:30) — the source AG was never torn down, so it is still fully consistent and current as of T+00:00.
2. Repoint the DNS CNAME / connection string back to the source AG listener.
3. Re-enable write traffic at the app tier.
4. Run the same smoke test suite against the source to confirm normal service resumed.
5. Declare rollback complete on the bridge call; open an incident record capturing the drift/failure evidence from the aborted cutover for root-cause analysis before the next attempt.

WHY ROLLBACK IS FAST

Because the source was only write-frozen, never decommissioned or resynced backwards, rollback is the same repoint operation as cutover, just aimed the other direction — typically under 3 minutes, well inside the original 15-minute budget.

9. Reconciliation Scripts

9.1 Row Count Comparison

```
-- Run on BOTH source and target; compare output externally or via linked server
SELECT
    s.name AS schema_name,
    t.name AS table_name,
    SUM(p.rows) AS row_count
FROM sys.tables t
JOIN sys.schemas s ON t.schema_id = s.schema_id
JOIN sys.partitions p ON t.object_id = p.object_id AND p.index_id IN (0,1)
GROUP BY s.name, t.name
ORDER BY row_count DESC;
```

9.2 Checksum Comparison (Per Table)

```
-- CHECKSUM_AGG over BINARY_CHECKSUM(*) gives an order-independent table fingerprint
SELECT CHECKSUM_AGG(BINARY_CHECKSUM(*)) AS table_checksum
FROM dbo.Orders; -- repeat per critical table, source vs target
```

9.3 Log-Shipping / CDC Lag Snapshot

```
-- Option B: on the monitoring server
EXEC sp_help_log_shipping_monitor;

-- Option A: DMS task metrics via CLI
aws dms describe-replication-tasks \
    --filters Name=replication-task-id,Values=contosoorders-cdc-task \
    --query 'ReplicationTasks[0].ReplicationTaskStats'
```

9.4 Automated Drift Alert (Daily During Phase 2)

Wrap 9.1 and 9.2 into a SQL Agent job that runs against both source and target via a linked server or a lightweight PowerShell/dbatools comparison, writes results to a DriftLog table, and emails on any non-zero delta. This is what gives you the confidence to sign off the exit criteria in Section 5.

10. Timing Budget Reference

Actual figures from rehearsal against the clone environment (Section 3.2). Use these to calibrate your own rehearsal, not as guaranteed numbers — network RTT, table count, and index maintenance all shift them.

Activity	Elapsed Time	Blocking the Outage Window?
Full backup + compression (4 TB source)	~2 hr 40 min	No — Phase 1, source stays live
Transfer to target over Direct Connect	~3 hr 10 min	No — Phase 1
Restore WITH NORECOVERY on target	~2 hr 05 min	No — Phase 1
Log-ship catch-up to steady state	~6–10 hr (one-time)	No — Phase 2
Ongoing log-ship / CDC cadence	1–5 min lag, continuous	No — Phase 2
Final log flush + zero-lag confirm	1–2 min	Yes — T+01:00 to T+02:30
Reconciliation (row count + checksum)	1–2.5 min	Yes — T+03:00 to T+05:30
Recovery + AG join / RDS available check	1–1.5 min	Yes — T+05:30 to T+06:30
DNS repoint propagation	30–60 sec (low TTL)	Yes — T+08:30 to T+09:30
Smoke tests	1–1.5 min	Yes — T+09:30 to T+11:00
Total inside outage window	~11–14 min	Target: under 15 min

11. Troubleshooting & Common Pitfalls

Log-shipping restore falls behind and never catches up

- Check target disk I/O latency (sys.dm_io_virtual_file_stats) — io2/gp3 volumes under-provisioned for restore throughput is the most common cause.
- Confirm the restore job isn't serialized behind an antivirus scan or backup agent locking the .trn files on arrival.

DMS CDC task shows high CDCLatencyTarget but low CDCLatencySource

- Source-side capture is fine; the target (RDS) apply is the bottleneck — check RDS instance CPU/IOPS, and consider a larger DMS replication instance or parallel apply threads.

Reconciliation checksum mismatch on a specific table only

- Check for a rowversion/timestamp or identity column with different seed/increment on the target — these should be excluded from checksum comparison or reseeded to match exactly before cutover.
- Check for a nightly maintenance job (e.g., index rebuild changing page order) that ran on one side but not the other between reconciliation runs — rerun immediately before Phase 4 to eliminate timing skew.

Application still connects to old IP after DNS repoint

- Confirm TTL was actually lowered 24+ hours in advance, not just at cutover time — cached resolvers honor the TTL that was in effect when they cached the record.
- Check connection pooling at the app tier; some drivers cache resolved IPs independently of OS DNS cache and need a pool recycle, not just a DNS change.

SQL Agent jobs on target fail immediately after cutover

- Server-level objects (job definitions, proxies, credentials, linked servers) are not part of a database restore — confirm these were scripted and applied to the target during Phase 1 (Section 3.1, step 7), not assumed to 'come with' the database.

12. Appendix — Supporting Scripts

12.1 Pre-Flight Connectivity Check (PowerShell)

```
Test-NetConnection -ComputerName EC2SQL01 -Port 1433
Test-NetConnection -ComputerName EC2SQL02 -Port 1433
Test-NetConnection -ComputerName vmsql-listener -Port 1433
```

12.2 Active Write-Transaction Check Before Freeze

```
SELECT r.session_id, r.status, t.name AS transaction_name,
       DATEDIFF(SECOND, t.transaction_begin_time, GETDATE()) AS duration_sec
FROM sys.dm_tran_active_transactions t
JOIN sys.dm_tran_session_transactions st ON t.transaction_id = st.transaction_id
JOIN sys.dm_exec_sessions r ON st.session_id = r.session_id
WHERE r.is_user_process = 1;
```

12.3 Deny/Allow Connect Toggle (Freeze / Unfreeze)

```
-- Freeze
ALTER LOGIN [AppServiceAccount] DISABLE;

-- Unfreeze (rollback or post-cutover on new primary)
ALTER LOGIN [AppServiceAccount] ENABLE;
```

12.4 DMS CLI Snippets

```
aws dms create-replication-task \
  --replication-task-identifier contosoorders-cdc-task \
  --source-endpoint-arn arn:aws:dms:us-east-1:...:endpoint:SRC \
  --target-endpoint-arn arn:aws:dms:us-east-1:...:endpoint:TGT \
  --replication-instance-arn arn:aws:dms:us-east-1:...:rep-instance \
  --migration-type cdc \
  --cdc-start-position 'checkpoint:00000000000000000000'
```